1. DATA STRUCTURES AND ALGORITHIMS

- **Data Structures**

A **data structure** is a way of organizing and storing data so that it can be used efficiently. Different data structures are designed to store and manage data in different ways. Examples include arrays, linked lists, stacks, queues, and trees.

According to W3Schools, arrays are a type of data structure used to store multiple elements. Data structures are important because algorithms use them to store and work with data.

- **Algorithms**

An **algorithm** is a set of instructions or steps used to solve a problem or perform a specific task. Algorithms can be used to process, search, sort, and manipulate data.

In **Data Structures and Algorithms**, data structures and algorithms work together. The data structure determines how information is stored, while the algorithm determines how the information is processed or used.

2. LINEAR DATA STRUCTURE

A **linear data structure** is a type of data structure where elements are arranged in a sequential or linear order. The elements are organized so that they can be processed one after another.

Examples of linear data structures include:

- Arrays
- Linked Lists
- Stacks
- Queues

A linear data structure is useful when data needs to be organized in a specific sequence. An array is one of the basic examples of a linear data structure.

3. ONE DIMENSION ARRAY

A **one dimensional array** is a linear data structure that stores multiple elements in a sequence. Each element can be accessed using its position or index.

For example:

Numbers = [10, 20, 30, 40, 50]

The array can be represented as:

Index: 0 1 2 3 4

Value: 10 20 30 40 50

The first element has an index of 0, the second element has an index of 1, and so on.

For example:

Numbers[0] = 10

Numbers[2] = 30

Numbers[4] = 50

According to W3Schools, an array is a data structure used to store multiple elements. Arrays are also commonly used by algorithms for tasks such as searching and sorting.

4. THE BIG O NOTATION

Big O notation is a mathematical notation used in computer science to describe the time complexity of an algorithm. It describes how the running time of an algorithm increases as the amount of data, represented by **n**, increases.

Some common examples of Big O notation are:

- **O(1)** – Constant time
- **O(log n)** – Logarithmic time
- **O(n)** – Linear time
- **O(n^2)** – Quadratic time

For example, an algorithm that needs to check every value in an array may have a time complexity of **O(n)** because the number of operations increases as the number of values increases.

Big O notation is useful for comparing algorithms and understanding how efficiently they can handle larger amounts of data. W3Schools explains that Big O is commonly used to describe the worst-case time complexity of an algorithm.

5. BASIC SEARCHING ALGORITHIM

A **searching algorithm** is an algorithm used to find a particular value within a collection of data. Searching algorithms can be used to determine whether a specific value exists and, in many cases, identify its position or index.

Two basic searching algorithms are **Linear Search** and **Binary Search**.

6. LINEAR SEARCH

Linear Search, also called sequential search, is a simple searching algorithm that checks each element one by one.

For example, if we have:

[10, 25, 30, 45, 50]

and we want to find **45**, Linear Search checks the values in order:

10 → Not found

25 → Not found

30 → Not found

45 → Found

How it Works

1. Start at the first element of the array.
2. Compare the element with the target value.
3. If the value matches the target, the search is successful.
4. If it does not match, move to the next element.

5. Continue until the value is found or the end of the array is reached.

According to W3Schools, Linear Search goes through an array value by value from the beginning and compares each value with the value being searched for.

The worst-case time complexity of Linear Search is **O(n)** because the algorithm may need to check all n elements.

One advantage of Linear Search is that the array does **not need to be sorted**.

7. BINARY SEARCH

Binary Search is a searching algorithm that repeatedly checks the middle element of a **sorted array**. After comparing the middle element with the target, it eliminates approximately half of the remaining search area.

For example:

[10, 20, 30, 40, 50, 60, 70]

If we want to find **60**, Binary Search first checks the middle value:

[10, 20, 30, 40, 50, 60, 70]

↑

40

Since 60 is greater than 40, the algorithm searches only the right side:

[50, 60, 70]

It then checks the middle value of this smaller section:

[50, 60, 70]

↑

60

The value 60 is found.

How it works:

1. Start with a sorted array.
2. Find the middle element.
3. Compare the middle element with the target.
4. If they are equal, the target has been found.
5. If the target is smaller, search the left half.
6. If the target is larger, search the right half.
7. Continue until the target is found or there are no more elements to search.

According to W3Schools, each time Binary Search checks a new value, the search area is approximately cut in half. Because of this, its time complexity is **O(log n)**.

Binary Search requires the array to be **sorted**, unlike Linear Search.

DIFFERENCE BETWEEN LINEAR SEARCH AND BINARY SEARCH

Linear Search checks each item one by one until the target is found. It does not require the data to be sorted.

Binary Search checks the middle item first and then searches either the left or right half. It requires the data to be sorted.

In simple terms:

Linear Search = checks one by one.

Binary Search = checks the middle and divides the search in half.

REFERENCES:

https://www.w3schools.com/

https://www.w3schools.com/dsa/dsa_timecomplexity_theory.php

https://www.w3schools.com/dsa/dsa_data_arrays.php

https://www.w3schools.com/dsa/dsa_algo_linearsearch.php